

Manuale per Sviluppatori: ClubManager

Documentazione Tecnica e Architettura di Sistema

1. Introduzione e Stack Tecnologico

ClubManager è un'applicazione web server-side progettata per la gestione delle società calcistiche. L'architettura è basata sul pattern architetturale procedurale/modulare, ottimizzata per ambienti LAMP/WAMP.

Stack utilizzato:

- **Backend:** PHP (≥ 7.4 / 8.x)
 - **Database:** MySQL / MariaDB
 - **Frontend:** HTML5, CSS3 nativo (variabili CSS, Flexbox/Grid), FontAwesome 6 per l'iconografia.
 - **Testing:** PHPUnit per i test di unità.
-

2. Struttura delle Directory

Il repository è organizzato per separare la logica applicativa dai test e dalla documentazione:

Plaintext

/CLUB-MANAGER/

```
|— public/           # Root directory del Web Server (Document Root)
|   |— connessione.php # Modulo di connessione al Database
|   |— dashboard.php  # Core HUB (Dashboard)
|   |— salva_*.php    # Controller per l'elaborazione dei form (POST)
|   |— gestione_*.php # Viste per l'elenco e la lettura dei dati (GET)
|   |— ...            # Altri moduli applicativi
|— phpunit/          # Suite di Unit Testing
|   |— ControlloAllenamentoTest.php # Test per la logica anti-sovrapposizione
|— clubmanager.sql    # Dump completo del database (Schema + Dati)
|— README.md          # Presentazione e requisiti di base
|— Manuale_Utente.pdf  # Istruzioni lato end-user
|— Manuale_Sviluppatore.md # Questa documentazione
```

3. Pattern di Connessione al Database

(**connessione.php**)

Per garantire la massima compatibilità tra i vari moduli (alcuni dei quali legacy o derivati da librerie esterne), il file `connessione.php` implementa un **Adapter Universale**. All'inclusione del file, il sistema istanzia simultaneamente due oggetti di connessione globali:

1. `$pdo` (Classe `PDO`): Utilizzato prevalentemente per i moduli di autenticazione (`login.php`).
 2. `$mysql` (Classe `mysqli`): Utilizzato per l'estrazione rapida e l'elaborazione dei dati nella Dashboard e nelle viste CRUD.
-

4. Sicurezza e Controllo Accessi (RBAC)

Il sistema implementa un *Role-Based Access Control* gestito tramite sessioni PHP (`$_SESSION`). Ogni pagina protetta deve iniziare con il seguente middleware di blocco:

```
PHP
session_start();
require "connessione.php";
if (!isset($_SESSION['user_id'])) { header("Location: login.php"); exit; }
```

L'autorizzazione granulare all'interno delle pagine è basata sulla variabile `$ruolo_id` (1 = Admin, 2 = Allenatore, 3 = Visualizzatore), che attiva o nasconde determinati nodi del DOM (es. form di inserimento).

5. Architettura del Database e Ottimizzazione (Viste)

Per ridurre il carico elaborativo del backend e semplificare le query nei file PHP, la business logic dell'aggregazione dati è stata demandata al motore del Database tramite le **VIEW**.

Le viste principali che lo sviluppatore deve interrogare sono:

- `vista_giocatori_squadre`: Risolve le tabelle ponte restituendo l'atleta completo di squadra e categoria.
 - `vista_calendarioAllenamenti`: Unisce `allenamenti`, `squadre`, `campi` per il rendering diretto del planning.
 - `vista_documenti_in_scadenza`: Pre-filtra le date utilizzando funzioni native (`DATE_ADD(CURDATE(), INTERVAL 30 DAY)`).
-

6. Logica Core: Sistema Anti-Sovrapposizione (RF5.4)

La funzione più critica del sistema si trova in `salvaAllenamento.php`. L'algoritmo previene le doppie prenotazioni di un campo valutando l'intersezione temporale. Due eventi si sovrappongono se e solo se l'inizio del nuovo evento precede la fine del vecchio **E** la fine del nuovo evento segue l'inizio del vecchio:

SQL

```
SELECT id FROM allenamenti  
WHERE campo_id = ? AND data = ?  
AND (ora_inizio < ? AND ora_fine > ?)
```

7. Debito Tecnico e Sviluppi Futuri (Gestione TODO)

In conformità con il ciclo di sviluppo concordato, le interfacce per le operazioni di Update (Modifica) e Delete (Cancellazione) sono state posticipate alla successiva release. Gli sviluppatori che prenderanno in carico il progetto troveranno nel codice sorgente i marcatori standard `// TODO (RFx.x)` o `` posizionati nei punti esatti in cui iniettare la logica mancante.

Priorità per la prossima Sprint:

1. Implementazione CRUD completo per il modulo `gestione_campi.php`.
2. Sviluppo logica di file upload (`multipart/form-data`) in `documenti.php`.
3. Inserimento di grafici analitici (es. Chart.js) nella `dashboard.php`.